Unsupervised Learning: Clustering

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang

From Lecture 21

Unsupervised learning II: Manifold Learning

- *Visualizing nonlinear structure with t-SNE and UMAP*
- *Measuring similarity and distance between data points*
- *Grouping unlabeled data using clustering methods*

In this lecture, you will learn

- *Partition-based clustering with K-means and K-medoids*
- *Choosing the number of clusters using elbow and silhouette methods*
- *Density-based clustering with DBSCAN*

Clustering: Distance between numerical feature vectors

To perform clustering, we must have a measure of similarity or distance between data points

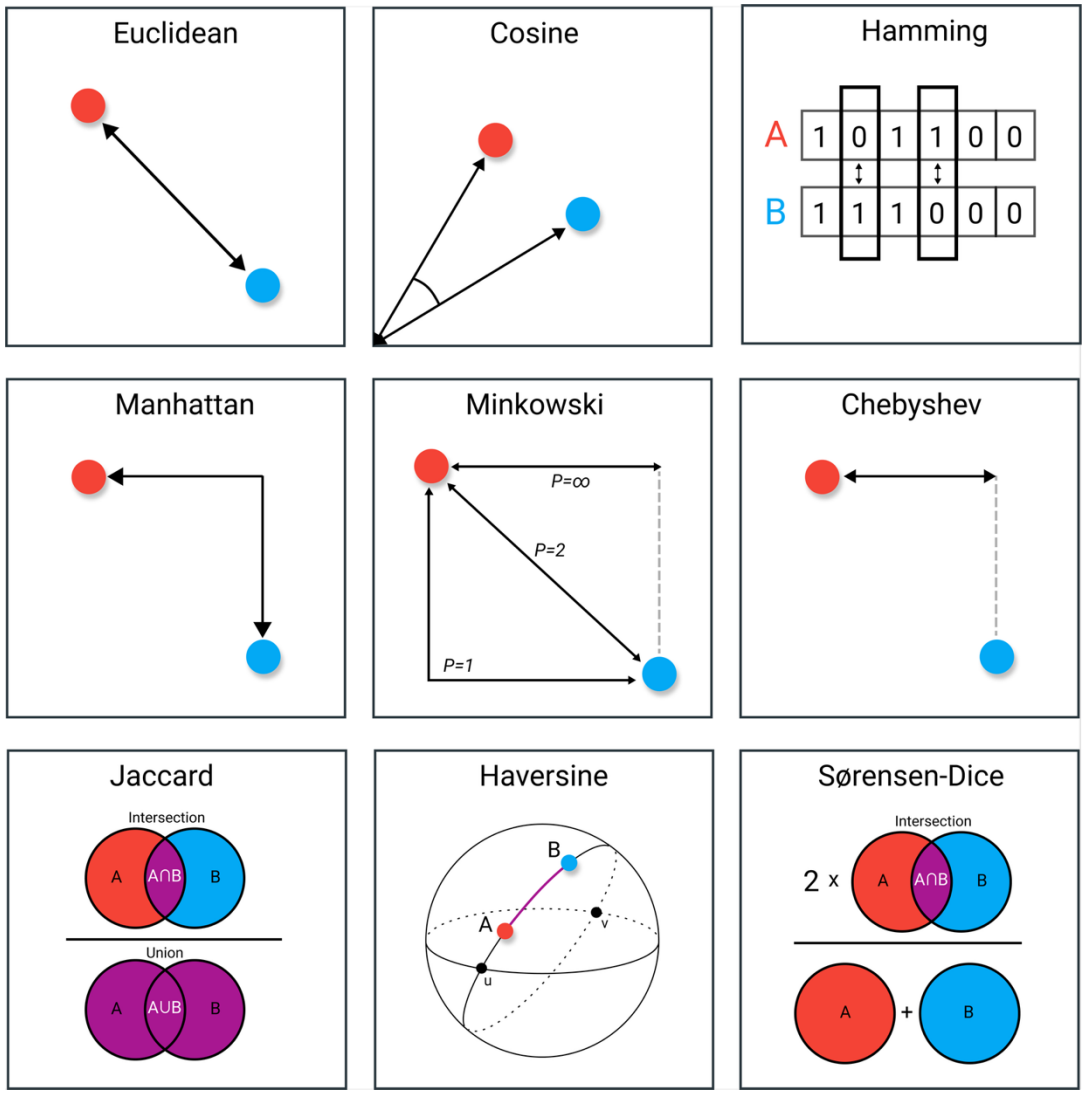
General Minkowski distance

$$d(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{m=1}^{d} |x_{im} - x_{jm}|^r \right)^{1/r}$$

- $\boldsymbol{x}_i, \boldsymbol{x}_j$= two samples
- x_{im}= feature m of sample i
- d = number of features
- r = parameter controlling the definition of distance

r	1	2	∞
Distance	Manhattan	Euclidean	Chebyshev

Max value



Maarten Grootendorst @ medium.com

Similarity for Fingerprints and Sets

To perform clustering, we must have a measure of similarity or distance between data points

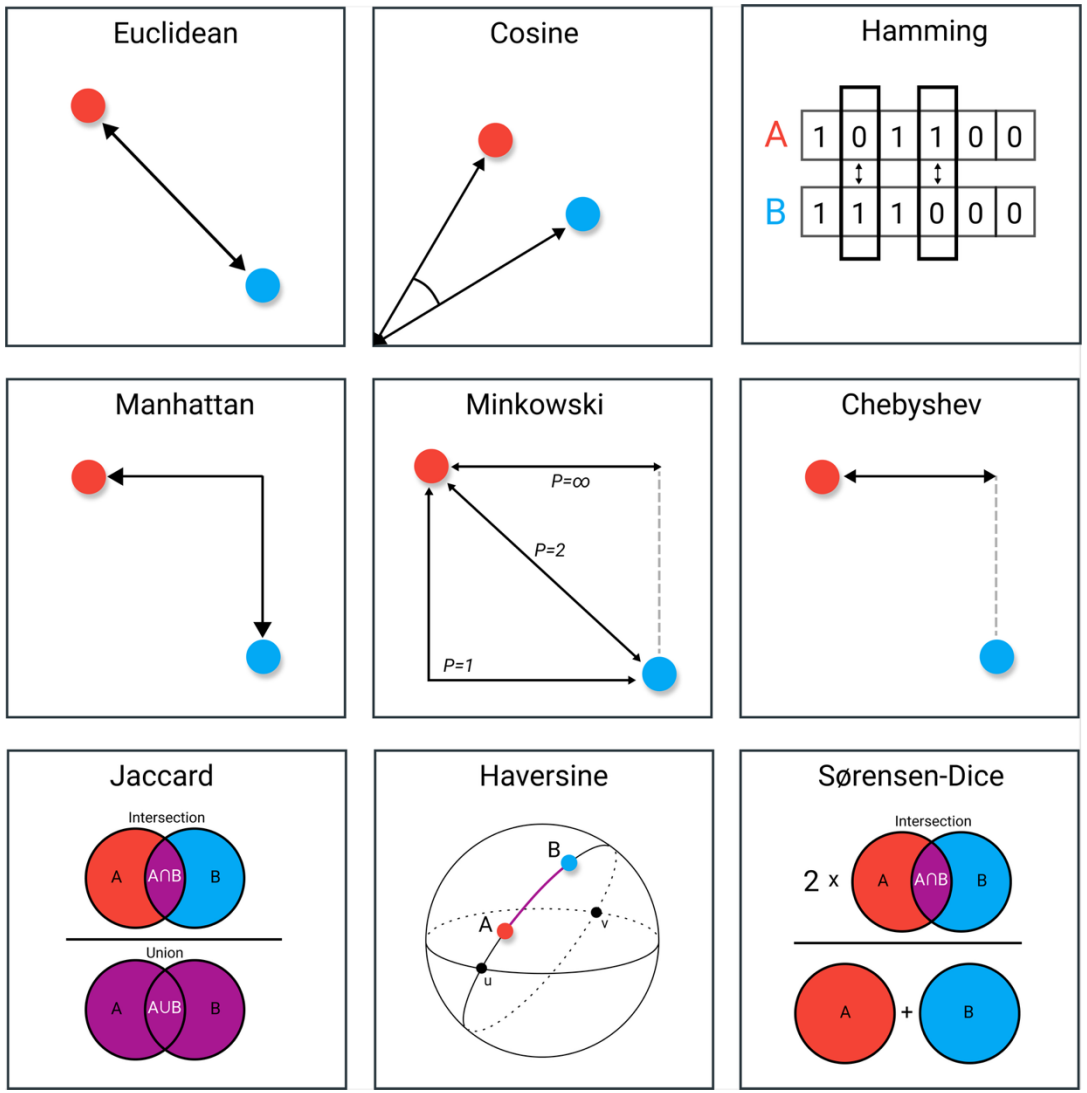
Jaccard similarity for sets

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- $A \cap B$ = features present in **both**
- $A \cup B$ = features present in **either**
- $J = 1$ means identical
- $J = 0$ means no overlap

$$A = \{a, b, c, d\}, \qquad B = \{b, c, e\}$$

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{2}{5} = 0.4$$



Maarten Grootendorst @ medium.com

Similarity for Fingerprints and Sets

To perform clustering, we must have a measure of similarity or distance between data points

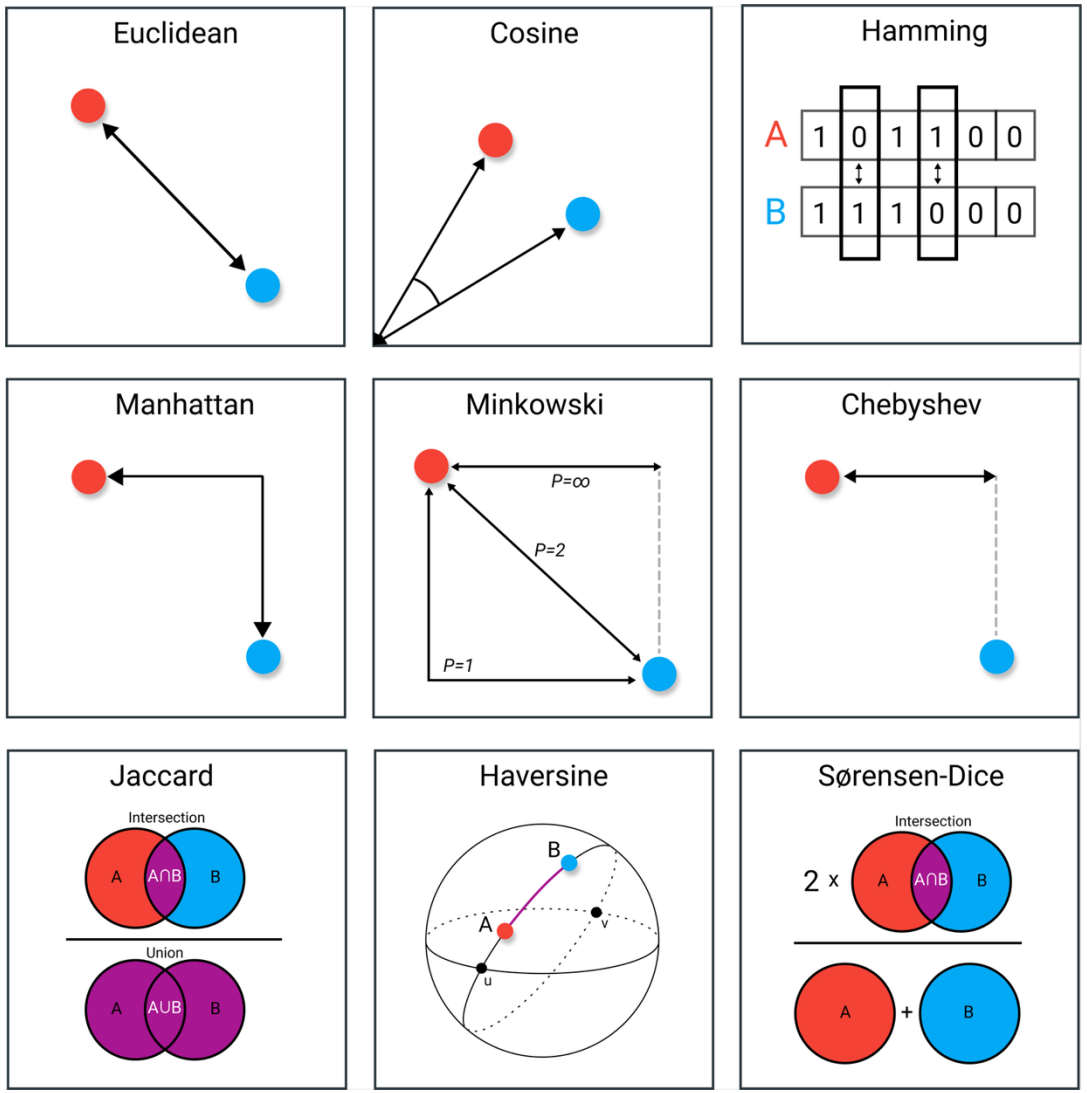
Tanimoto similarity

$$T(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{x}_i^T \mathbf{x}_j}{\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - \mathbf{x}_i^T \mathbf{x}_j}$$

For binary fingerprints, Tanimoto similarity is equivalent to Jaccard similarity.

$$\mathbf{x}_i = [1, 0, 1, 1, 0]$$
$$\mathbf{x}_j = [1, 1, 1, 0, 0]$$

$$T = \frac{2}{4} = 0.5$$



Maarten Grootendorst @ medium.com

Similarity for Fingerprints and Sets

To perform clustering, we must have a measure of similarity or distance between data points

Cosine similarity

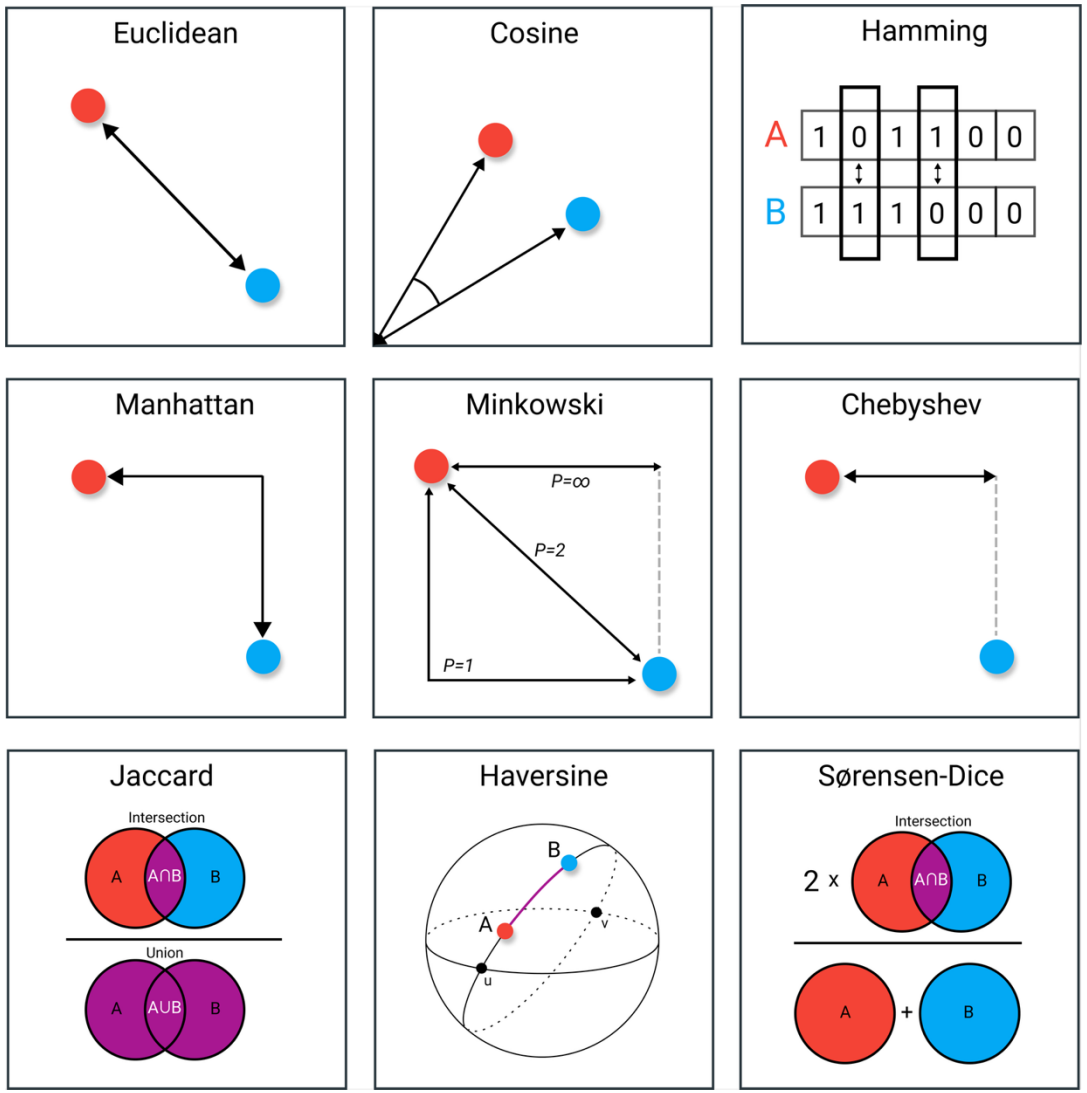
- Measures the *angle between two vectors*, largely ignoring their magnitude. Common for text, embeddings, and high-dimensional descriptors.
- **1 = same direction, 0 = orthogonal.**

Hamming distance

- Counts *how many positions are different* between two equal-length vectors or strings. Useful for binary fingerprints, sequences, and categorical data.
- Example: 10110 vs. 11100 differs at **2 positions**.

Haversine distance

- Measures the **shortest distance along the surface of a sphere** using latitude and longitude. Mainly used for geographic data.



Maarten Grootendorst @ medium.com

Categorization of Clustering Models

Partition Methods

Divide the data directly into a fixed number of clusters, usually by minimizing within-cluster dissimilarity.

Methods: K-means, k-medians, k-modes

Model-based Clustering

Assume the data are generated from a mixture of probability distributions and estimate which distribution generated each sample.

Methods: Gaussian Mixture Models

Hierarchical Methods

Build a hierarchy of clusters by repeatedly merging smaller groups or splitting larger groups.

Methods: Agglomerative clustering, divisive clustering, Ward linkage

Density Models

Identify clusters as connected regions of high point density and label sparse points as noise or outliers.

Methods: DBSCAN, HDBSCAN, OPTICS

K-means Clustering

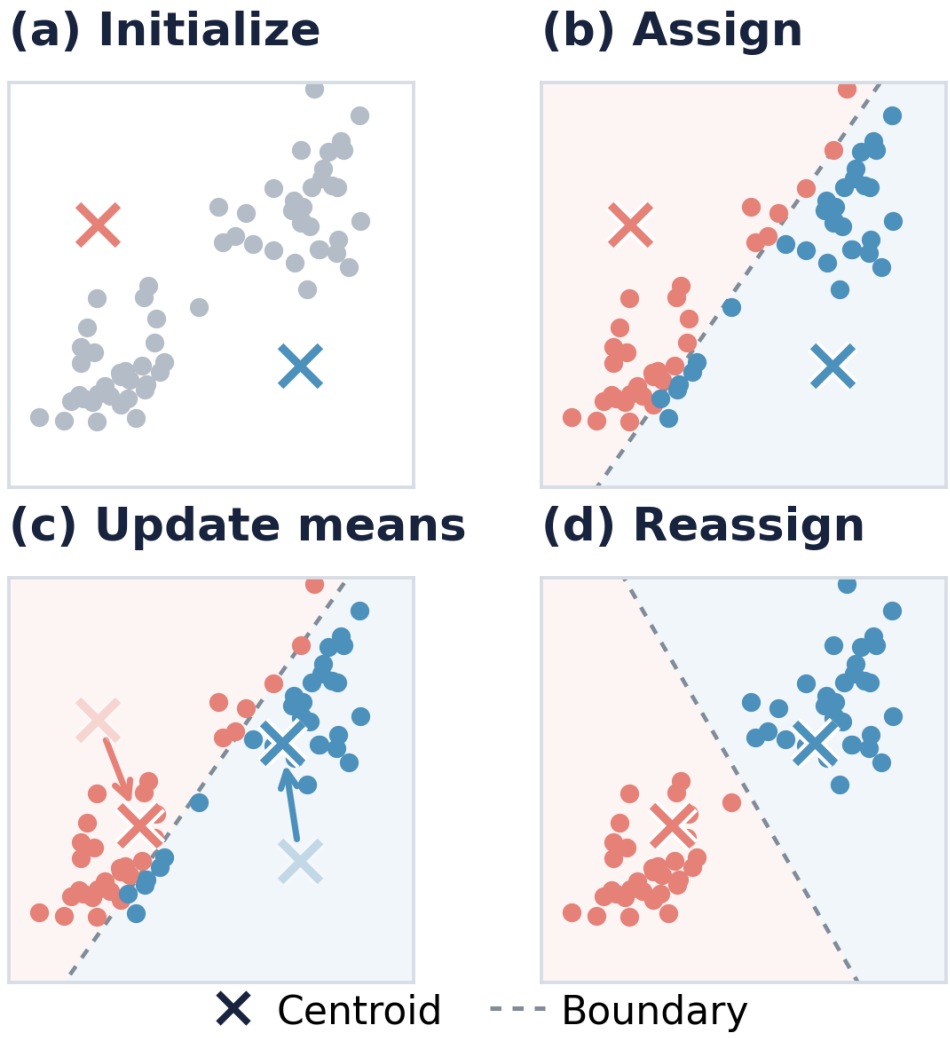
Goal: divide the data into K clusters so that points within each cluster are close to their cluster center.

1. Choose the number of clusters K
2. Initialize K cluster centers
3. Assign each point to its *nearest center*
4. Recompute each center as the *mean of its assigned points*
5. Repeat steps 3 and 4 until assignments stop changing

Cluster center:

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$$

- $\boldsymbol{x}_i$ = sample i
- C_k = samples assigned to cluster k
- $|C_k|$ = number of samples in cluster k
- $\boldsymbol{\mu}_k$= centroid of cluster k



What Is K-Means Optimizing?

K-means minimizes the *within-cluster sum of squared distances*:

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

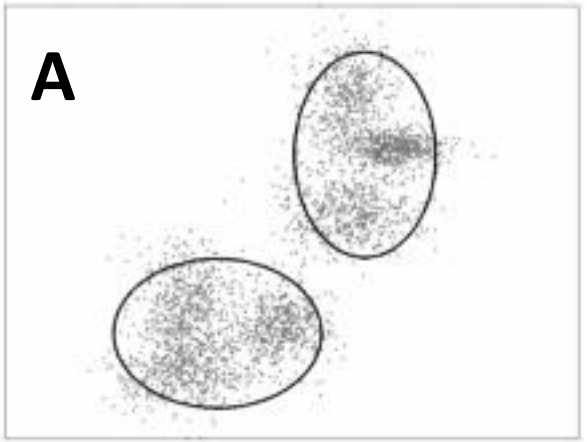
Make every point as close as possible to the center of its own cluster.

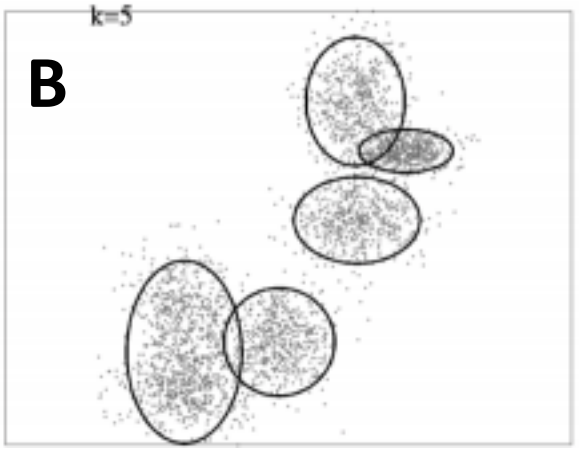
Works well	Struggle with
• Compact, roughly spherical clusters • Similar cluster sizes and densities • Numerical features with meaningful Euclidean distance	• Curved or irregular clusters • Very different cluster sizes or densities • Outliers • Poor feature scaling • Different initializations

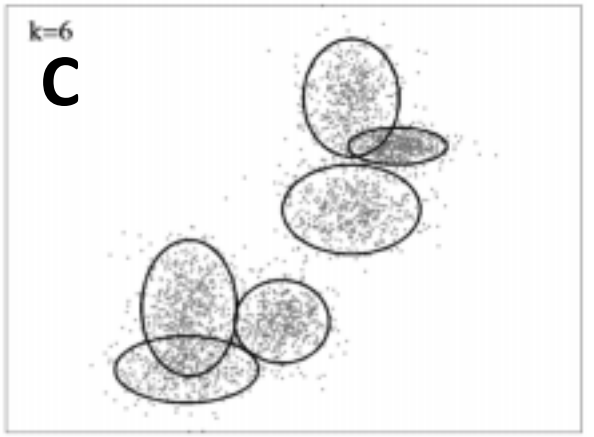
K-means is simple and fast, but its definition of a cluster is strongly tied to Euclidean distance and the mean.

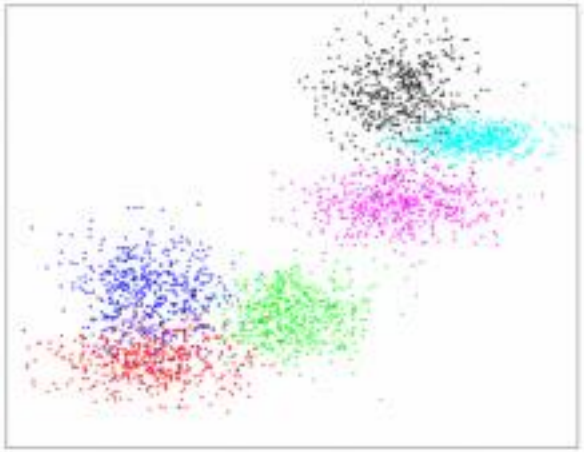
How Many Clusters Should We Use?

The number of clusters K is usually **unknown** and must be selected from the data and scientific context.



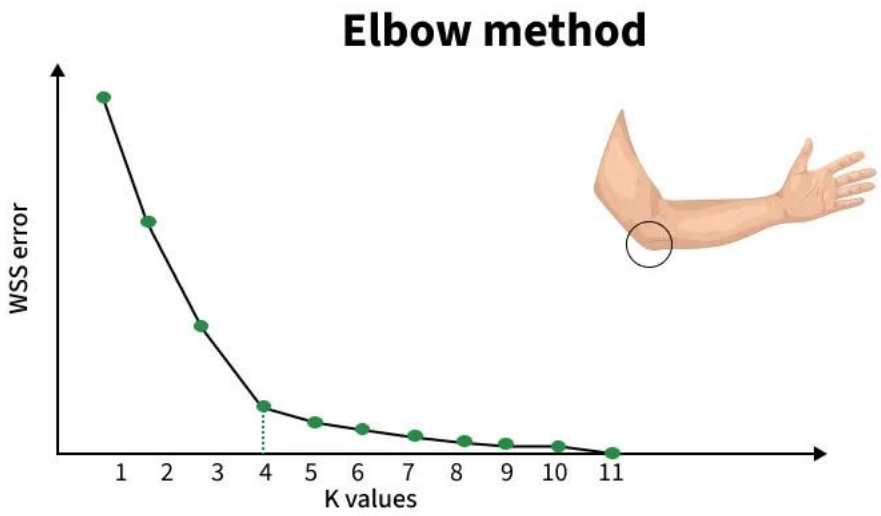






Elbow method

- Run K-means for several values of K and plot the *within-cluster sum of squares*.
- Choose K near the point where adding more clusters gives *diminishing improvement*.



Use the **within-cluster sum of squares**, WSS:

$$\mathrm{WSS}(K) = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

- K = number of clusters
- C_k = set of points assigned to cluster k
- $\boldsymbol{x}_i$ = data point i
- $\boldsymbol{\mu}_k$= centroid of cluster k

$$\Delta^2 W(K) = W(K+1) - 2W(K) + W(K-1)$$

Maximum curvature

Silhouette Score

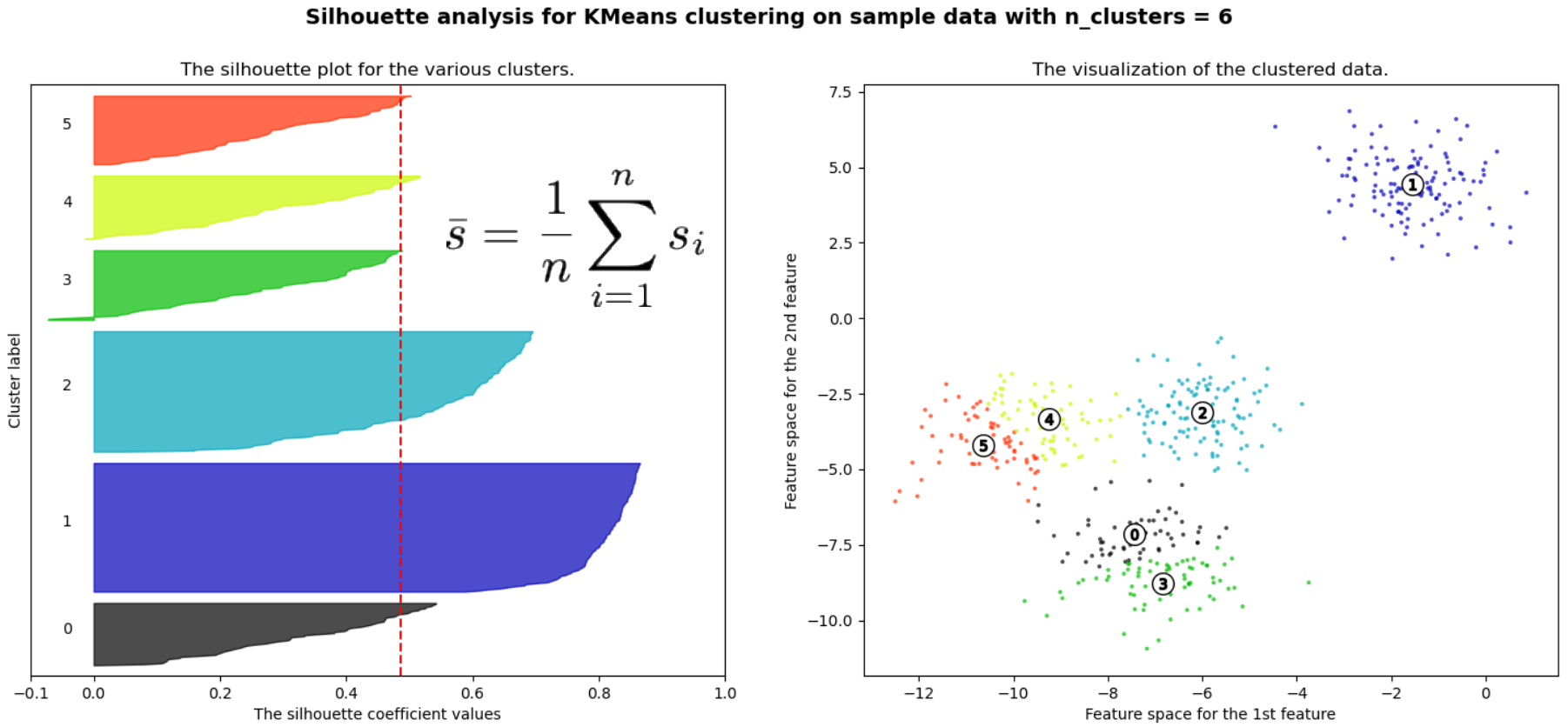
First, for each point i

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- a_i = average distance from point i to points in its *own cluster*
- b_i = smallest average distance from point i to points in *another cluster*
- s_i = silhouette score for point i

Interpretation

- $s_i \approx 1$ means *well matched to its own cluster*
- $s_i \approx 0$ means *between two clusters*
- $s_i < 0$ means it may be *assigned to the wrong cluster*



K-medoids Clustering

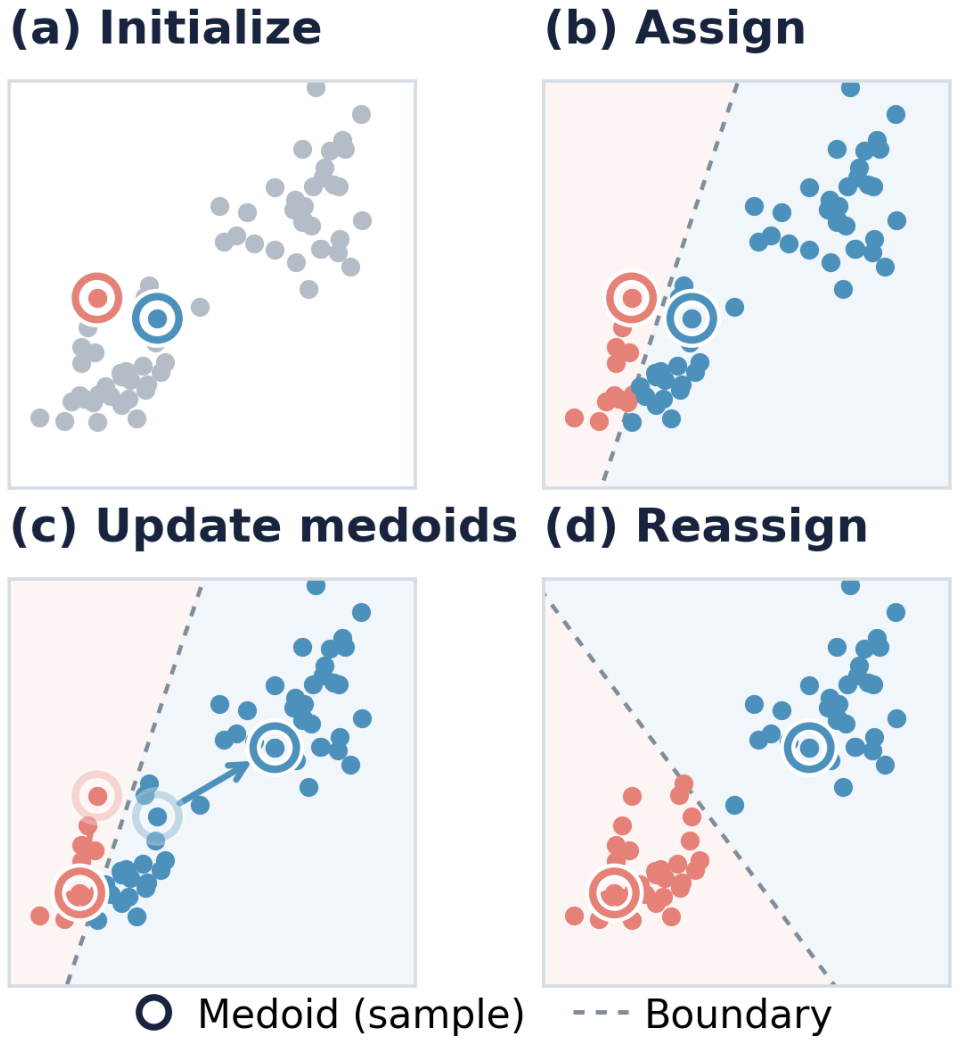
Goal: divide the data into K clusters so that points are close to a representative data point from their own cluster.

1. Choose the number of clusters K
2. Initialize K ***medoids from the observed samples***
3. Assign each point to its *nearest medoid*
4. Update each medoid to the point that *minimizes total distance within its cluster*
5. Repeat until the medoids and assignments stop changing

Cluster representative:

$$\mathbf{m}_k = \arg\min_{\mathbf{x}_j \in C_k} \sum_{\mathbf{x}_i \in C_k} d(\mathbf{x}_i, \mathbf{x}_j)$$

- $\boldsymbol{x}_i$ = sample i
- C_k = samples assigned to cluster k
- $d(\boldsymbol{x}_i, \boldsymbol{x}_j)$ = chosen distance or dissimilarity
- $\boldsymbol{m}_k$= medoid, an actual sample in cluster$k$



- *More robust to outliers than K-means*
- *Can use many different distance metrics, not only Euclidean*

Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Goal: identify clusters as *connected high-density regions* while labeling isolated points as noise.

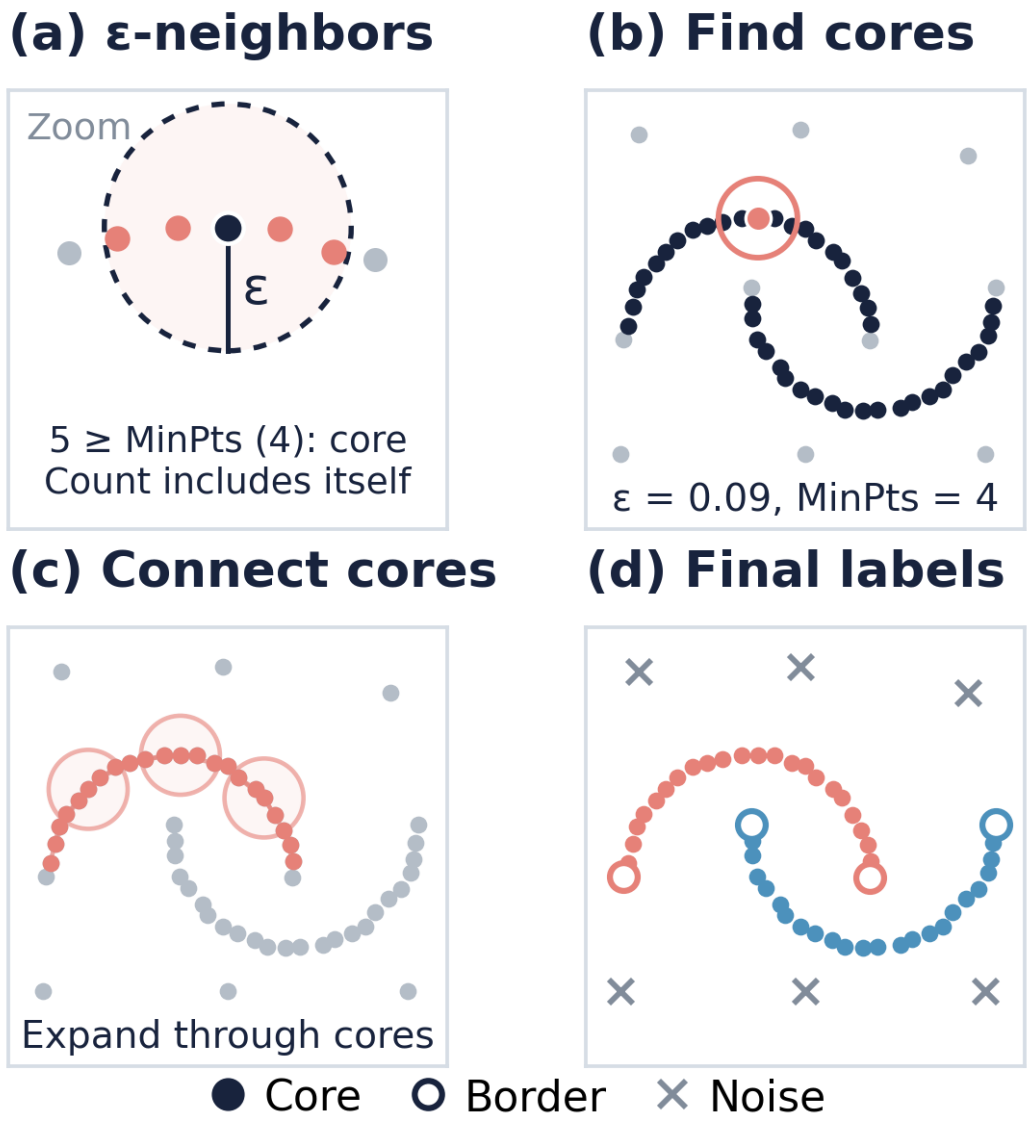
1. Choose the neighborhood radius ***epsilon*** and minimum number of points ***MinPts***
2. For each point, find all samples within distance epsilon

$$N_\varepsilon(\mathbf{x}_i) = \{\mathbf{x}_j : d(\mathbf{x}_i, \mathbf{x}_j) \leq \varepsilon\}$$

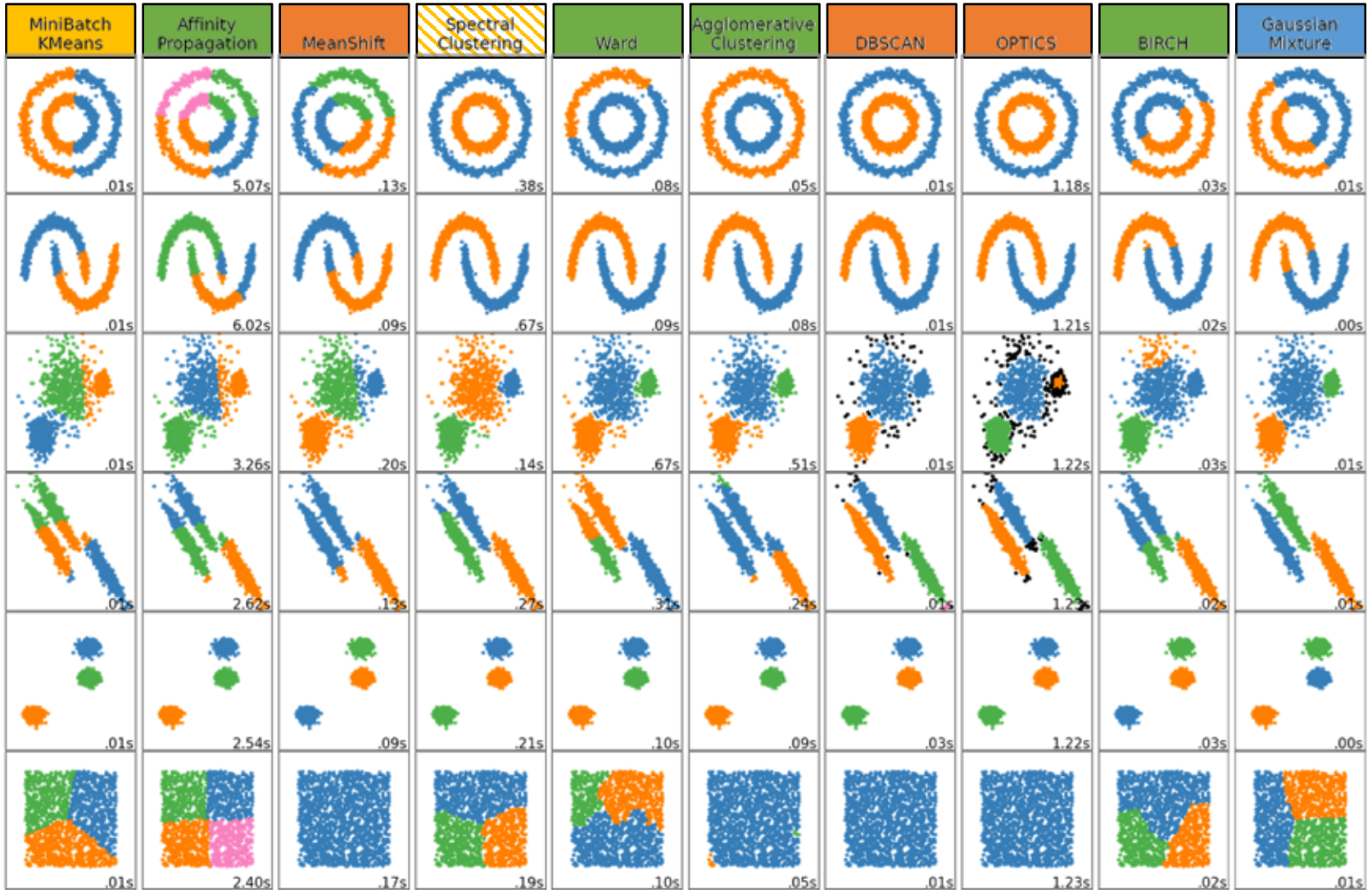
3. A point with at least MinPts neighbors becomes a ***core point***

$$|N_\varepsilon(\mathbf{x}_i)| \geq \text{MinPts}$$

4. Connect neighboring core points to grow a cluster
5. Assign nearby non-core points as ***border points***
6. Points that cannot be reached from any cluster are labeled ***noise***



There Is No Single Best Clustering Algorithm



https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html#sphx-glr-auto-examples-cluster-plot-cluster-comparison-py

Interactive Python Time

21_clustering.ipynb

In this lecture, we learned

- *Partition-based clustering with K-means and K-medoids*
- *Choosing the number of clusters using elbow and silhouette methods*
- *Density-based clustering with DBSCAN*

Next lecture:

- *Explainable AI:*
  - *Permutation importance*
  - *PDP and ICE*
  - *SHAP analysis*